

SYNOPSIS OF THE B.TECH PROJECT ENTITLED

**DESIGN AND DEVELOPMENT OF A
SELF-CORRECTING RETRIEVAL-AUGMENTED
GENERATION (RAG) SYSTEM**

A synopsis Submitted
In Partial Fulfillment of the Requirements
for the Degree of
BACHELOR OF TECHNOLOGY
(Computer Science & Engineering)

By

Student Name	Enrollment Number
Manas Khanna	A25305223002
Gurmukh Singh	A25305223008
Arshleen Kaur	A25305223141

Under the Supervision of
Dr. Gurpreet Singh
Assistant Professor



AMITY UNIVERSITY PUNJAB, INDIA

July, 2026

Declaration

We hereby declare that this SYNOPSIS report entitled, “**Design and Development of a Self-Correcting Retrieval-Augmented Generation (RAG) System**”, being submitted by us to **Amity University Punjab** for the award of the degree of Bachelor of Technology (Computer Science & Engineering) is a record of bonafide work carried by us. We worked under the guidance and supervision of **Dr. Gurpreet Singh, Assistant Professor**.

The matter embodied in this report has not been submitted to any other university or institute earlier for the award of any degree or diploma.

Manas Khanna

B. TECH. Sem VII

(Session 2023-2027)

Roll No: A25305223002

Gurmukh Singh

B. TECH. Sem VII

(Session 2023-2027)

Roll No: A25305223008

Arshleen Kaur

B. TECH. Sem VII

(Session 2023-2027)

Roll No: A25305223141

Acknowledgements

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend our sincere thanks to all of them.

We would like to take the opportunity to express our humble gratitude to **Dr. Gurpreet Singh, Assistant Professor**, under whom we executed this project, and are most thankful for his constant guidance and willingness to share his vast knowledge which made us understand this project and its manifestation in great depth and helped us to complete the assigned task.

PLACE: Mohali, India

DATE: July 2026

Manas Khanna

Roll No: A25305223002

Gurmukh Singh

Roll No: A25305223008

Arshleen Kaur

Roll No: A25305223141

CONTENT

Declaration	i
Acknowledgements	ii
Abstract	1
1 Introduction	2
1.1 Literature Review	2
2 Scope and Objective of the Project	4
2.1 Project Scope	4
2.2 Project Objectives	4
3 Roles and Responsibilities	6
4 Requirements Analysis	7
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	7
5 Software Specification	8
6 System Design and Methodology	9
7 Use Case Diagram	10
8 Sequence Diagram	11
9 Data Flow Diagram	12
9.1 Level 0 DFD (Context Diagram)	12
9.2 Level 1 DFD	12
10 Entity Relationship Diagram (ERD)	13
11 Time Frame (Gantt Chart)	14
References	15

Abstract

Large Language Models (LLMs) have demonstrated exceptional capabilities in natural language understanding and text generation. However, they frequently suffer from factuality errors and hallucinations due to their reliance on static parametric knowledge. Retrieval-Augmented Generation (RAG) mitigates this by integrating external vector search. Nonetheless, standard RAG systems act as passive consumers of retrieved information, leaving them vulnerable to poor-quality retrievals, noise, and hallucinations if the retrieved documents are irrelevant.

This project proposes a **Self-Correcting Retrieval-Augmented Generation (Self-Correcting RAG)** system. The framework extends the Corrective Retrieval-Augmented Generation (CRAG) model by introducing a secondary self-evaluation and reflection module. A lightweight retrieval evaluator assesses retrieved document relevance, dynamically triggering knowledge refinement, web search augmentation, or query rewriting based on three confidence states: Correct, Ambiguous, and Incorrect. Following generation, a self-reflection loop critiques the answer’s groundedness, completeness, and hallucination status, triggering iterative context expansion or answer regeneration if required. This system aims to provide robust, factual, and verified responses for knowledge-intensive question-answering tasks.

1 Introduction

Large Language Models (LLMs) have transformed natural language processing. Despite their success, LLMs encounter factual errors, particularly when answering queries requiring specific, real-time, or long-tail knowledge. Retrieval-Augmented Generation (RAG) addresses this by fetching documents from an external corpus to aid the LLM.

Despite the benefits of RAG, standard implementations suffer from three critical challenges:

1. **Retrieval Failure:** If the retriever fetches irrelevant or noisy documents, the generator is misled, leading to incorrect answers.
2. **Redundant Context:** Complete documents are often fed to the generator, introducing clutter and exceeding the LLM’s optimal context length.
3. **Absence of Post-Generation Reflection:** The generator produces responses in a single forward pass without validating if the response is supported by the retrieved context.

To overcome these limits, this project presents a Self-Correcting RAG system. It introduces a two-tier evaluation mechanism: a *Pre-generation Retrieval Evaluator* that corrects retrieved contexts using internal refinement and external web search, and a *Post-generation Reflection Module* that evaluates the generated output for groundedness and completeness.

1.1 Literature Review

The evolution of knowledge-augmented generation spans several milestones:

- **Standard RAG (Lewis et al., 2020):** Introduced the framework coupling dense vector retrieval (e.g., DPR) with an encoder-decoder generator. It relies on the assumption that retrieved documents are always high-quality and relevant.
- **Self-RAG (Asai et al., 2024):** Proposed instruction-tuning LLMs with special “critique tokens” to decide when to retrieve and self-evaluate responses. Although powerful, Self-RAG requires fine-tuning large models (e.g., LLaMA-2 7B/13B), which is computationally expensive and complex to generalize across arbitrary LLMs.
- **Corrective RAG (CRAG) (Yan et al., 2024):** Introduced a lightweight retrieval evaluator (0.77B T5-large) that classifies retrieved relevance into *Correct*, *Incorrect*, and *Ambiguous*. It triggers web-search queries for incorrect/ambiguous cases and performs a decompose-then-recompose step on document segments to strip out noisy text.

Our project builds on the plug-and-play architecture of CRAG and integrates an orchestrator-driven self-correction feedback loop. Unlike CRAG, which assumes the generator output is final, our system adds a verification layer that critiques the response against the refined context, allowing self-corrective iterations before final delivery.

2 Scope and Objective of the Project

2.1 Project Scope

The scope of this project lies in designing and building a state-of-the-art framework for context-grounded text generation that functions as an intelligent, self-correcting agent. Specifically, it involves:

1. Structuring an ingestion pipeline that handles diverse text documents, builds vector index databases, and runs local dense retrievers.
2. Integrating a lightweight evaluator model that calculates relevance scores and outputs a routing state (Correct, Incorrect, or Ambiguous).
3. Constructing fallback pathways to query public search APIs for real-time information retrieval when the internal corpus is insufficient.
4. Designing an LLM-based critique module that scores generated responses for factual groundedness, relevance, and query coverage.

$$\text{Relevance Score} = f(\text{Retrieved Docs}, \text{Query})$$

5. Orchestrating these components as a directed graph using LangGraph to enable cyclical feedback loops.

2.2 Project Objectives

Given a user query x and a document corpus $\mathcal{C}$, standard RAG systems generate an answer y by directly feeding the retrieved documents $\mathcal{D}$ to the LLM generator. When $\mathcal{D}$ contains irrelevant information, the system fails. The objective is to design a self-correcting pipeline that:

1. Evaluates the quality of retrieved documents $\mathcal{D}$ before generation.
2. Refines or replaces $\mathcal{D}$ dynamically using document stripping or web search.
3. Critiques the generated answer y post-generation to check for hallucinations and incompleteness, regenerating the answer if confidence is low.

Formally, the retrieved documents are represented as:

$$\mathcal{D} = \{d_1, d_2, \dots, d_k\} \subset \mathcal{C}$$

The pre-evaluation score maps to a confidence state S :

$$S = \text{Evaluator}(x, \mathcal{D}) \in \{\text{Correct}, \text{Incorrect}, \text{Ambiguous}\}$$

For generating response y , the generator uses refined context K :

$$y = \text{LLM}(x, K)$$

Where the post-generation module validates the response by evaluating groundedness against the context:

$$\text{Groundedness}(y, K) \rightarrow \{\text{Entailed}, \text{Hallucinated}\}$$

3 Roles and Responsibilities

This project is a collaborative effort by the team members. To ensure efficient execution and integration, the project responsibilities are divided into major modules: Ingestion and Retrieval, Relevance Evaluation and Search Augmentation, and Critique Agents and Orchestration.

The development responsibilities are distributed as follows:

- **Module 1: Data Ingestion and Retrieval System**

Focuses on raw document processing, text splitting, metadata extraction, embedding generation, and indexing in the vector database. It ensures high-quality embedding representation of the internal knowledge base.

- **Module 2: Pre-generation Evaluation and Query Rewriting**

Handles the confidence-based relevance assessment of retrieved documents, query extraction, and external API search query generation. It is responsible for formatting documents into concise knowledge strips.

- **Module 3: Post-generation Reflection and Graph Orchestration**

Orchestrates the overall workflow using state-machine routing, executes the LLM generator, evaluates the generated output for groundedness and completeness, and manages the cyclical feedback loop.

All team members jointly participate in system integration, model evaluation, testing, benchmarking, and documentation.

4 Requirements Analysis

To develop a robust and effective Self-Correcting RAG system, the system requirements are categorized into functional and non-functional requirements:

4.1 Functional Requirements

1. **Vector Database Setup:** The system must load, chunk, embed, and store document corpora in ChromaDB/FAISS.
2. **Dense Retrieval:** The system must accept user queries and retrieve the top- k relevant documents.
3. **Relevance Evaluation:** A retrieval evaluator must assess the relevance of each document and classify the state into Correct, Ambiguous, or Incorrect.
4. **Web Search Fallback:** If the evaluation state is Incorrect or Ambiguous, the system must trigger query rewriting and perform web search queries to fetch external documents.
5. **Context Refinement:** Relevant retrieved context must be stripped of noise and formatted into concise knowledge strips.
6. **Answer Generation:** The LLM generator must generate responses using only the refined context.
7. **Post-Generation Reflection:** The feedback module must check the generated answer for groundedness (hallucinations) and query coverage, and trigger regeneration if necessary.

4.2 Non-Functional Requirements

1. **Accuracy and Faithfulness:** Responses must be highly faithful to the context, minimizing hallucinations.
2. **Performance and Latency:** The average processing time for a query (including retrieval, evaluation, search, and generation) must be optimized.
3. **Modularity:** The architecture must allow easy swap-in/swap-out of different embedding models or generators.
4. **Reliability:** The system must handle search API failures or model timeouts gracefully.

5 Software Specification

The software specification stack defined for the implementation of the Self-Correcting RAG project is shown in Table 1.

Specification Item	Details
Operating System	Arch Linux / Windows 11
IDE & Development Tools	Neovim / VS Code
Versioning System	Git & GitHub
Database	ChromaDB (Vector DB) & SQLite3 (State Store)
Web Server	FastAPI (Backend API)
Languages & Frameworks	Python 3.10+, LangGraph, LangChain

Table 1: Software Specifications.

6 System Design and Methodology

The proposed Self-Correcting RAG system is designed around an orchestrated workflow using LangGraph state machines. The detailed flow of the system is shown in Figure 1.

The system operates in four stages:

1. **Retrieval Stage:** Given a query x , the top- k documents are retrieved. The evaluator scores the documents. Let s_i be the relevance score of document d_i .
2. **Evaluator Stage:** Classification of document relevance.
 - **Correct:** If $\max(s_i) \geq T_{high}$, the documents are refined using document splitting into sentence-level strips. Low-relevance strips are filtered out.
 - **Incorrect:** If $\max(s_i) < T_{low}$, the documents are discarded. A query rewriter W extracts search keywords, and a Web Search API fetches external context.
 - **Ambiguous:** If $T_{low} \leq \max(s_i) < T_{high}$, internal documents are refined and combined with web search results.
3. **Generation Stage:** The LLM generator produces an initial answer A_0 from the refined context K .
4. **Critique Stage:** The reflection module evaluates NLI Groundedness (to prevent hallucinations) and Query Coverage. If A_0 fails verification, a correction node runs. It decides whether to fetch more information (triggering back-to-retriever loop) or to regenerate the response with a prompt highlighting the critique.

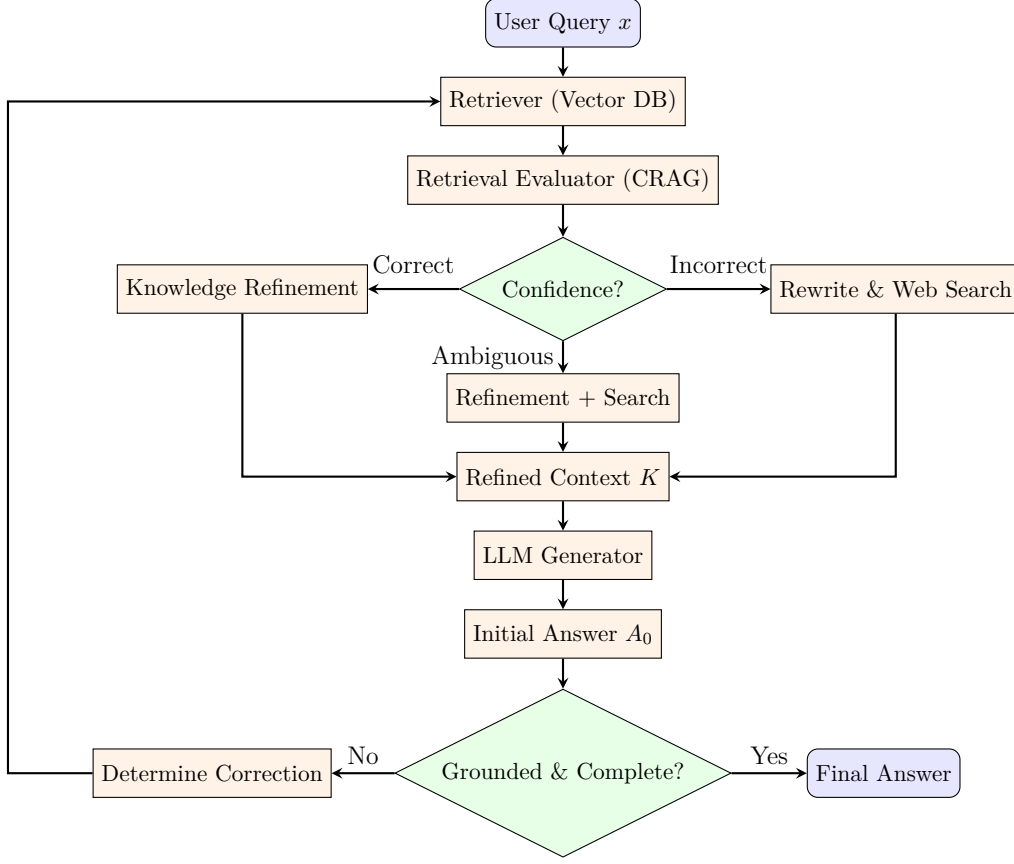


Figure 1: Proposed Self-Correcting RAG System Architecture.

7 Use Case Diagram

The Use Case Diagram defines the interactions between the external actors (User, Administrator) and the Self-Correcting RAG system. The diagram is illustrated in Figure 2.

The User primarily interacts with the query interfaces, while the Administrator configures vector database schemas, uploads reference files, and sets confidence scores for evaluation.

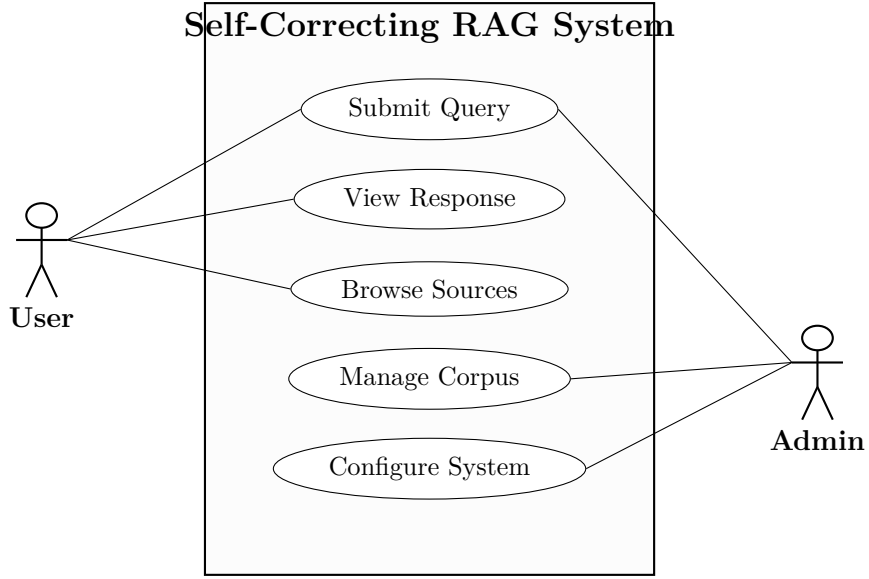


Figure 2: Use Case Diagram.

8 Sequence Diagram

The Sequence Diagram depicts the detailed step-by-step transaction flow and temporal execution between the User, Orchestrator, Retriever, Evaluator, Web Search API, Generator, and Critique modules. The process is shown in Figure 3.

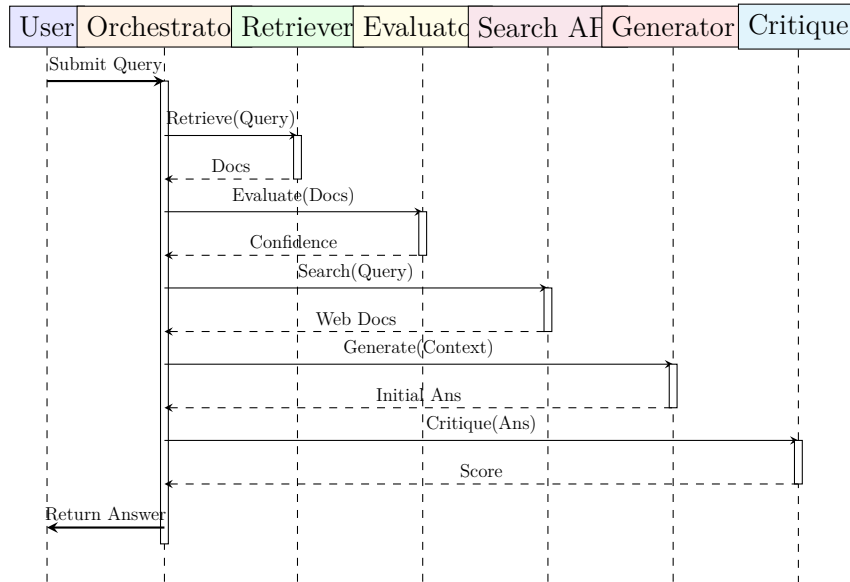


Figure 3: Sequence Diagram.

9 Data Flow Diagram

9.1 Level 0 DFD (Context Diagram)

The Context Diagram (Figure 4) outlines the scope of the system by illustrating the boundaries and external entities that supply information to, or consume outputs from, the RAG system.

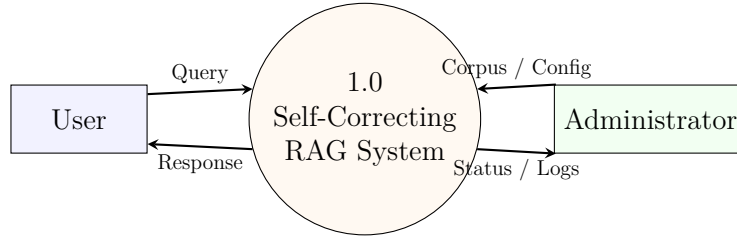


Figure 4: Level 0 DFD (Context Diagram).

9.2 Level 1 DFD

The Level 1 Data Flow Diagram (Figure 5) details the sub-processes responsible for document retrieval, confidence evaluation, web scraping fallback, context assembly, response generation, and post-generation reflection.

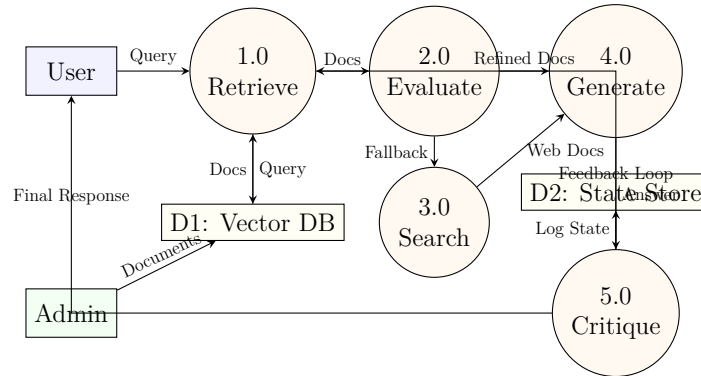


Figure 5: Level 1 DFD.

10 Entity Relationship Diagram (ERD)

To track the state of conversations, documents, system logs, evaluations, and web cache, a database schema is implemented. Figure 6 models this structure.

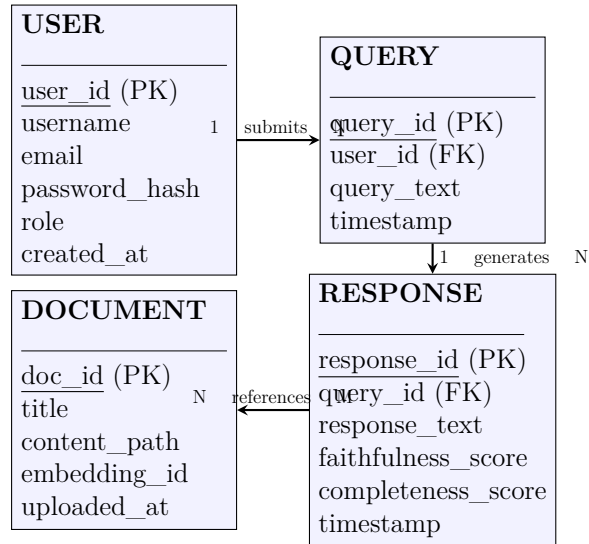


Figure 6: Entity Relationship Diagram.

11 Time Frame (Gantt Chart)

The project implementation is structured over a 16-week cycle during the final semester. The timeline distribution across key phases is shown in Figure 7.

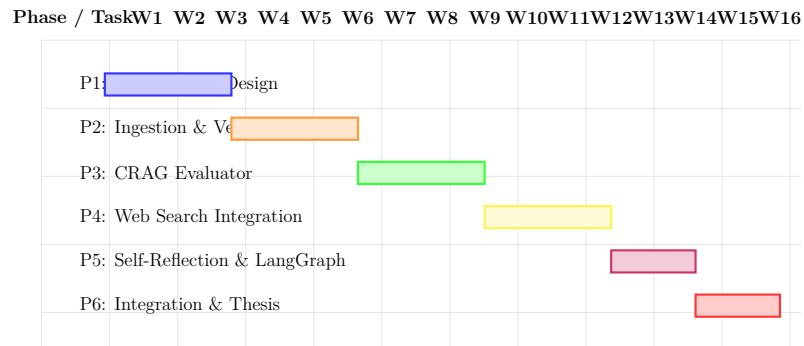


Figure 7: Project Gantt Chart Timeline.

References

- [1] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884*, 2024.
- [2] Akari Asai, Zequi Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [4] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys (CSUR)*, 55(12):1–38, 2023.
- [5] Harrison Chase. LangGraph: Building language agents as graphs. *LangChain Blog*, 2024.